

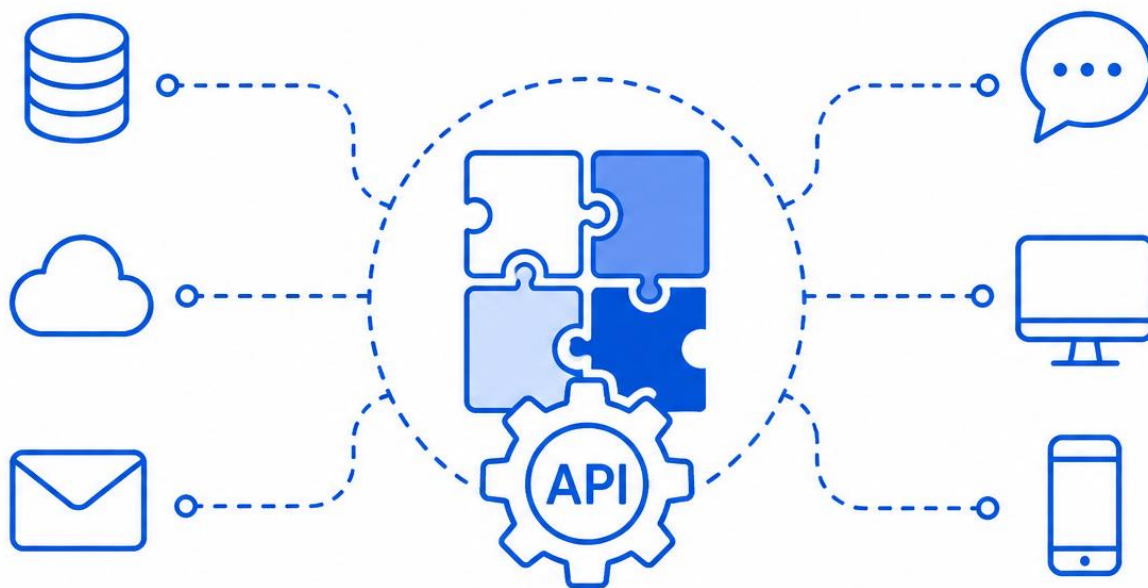


Services Computing (+SOAP)

Prof. Michele Ciavotta
Corso di Sistemi Distribuiti

Outline

- Introduzione ai servizi
- SOAP e WSDL
- REST (prossima lezione)



Architetture orientate ai servizi

Lo stile architetturale **a layer** è stato a lungo dominante perché favorisce la separazione delle responsabilità, strutturando il sistema in livelli gerarchici di astrazione e migliorando, in teoria, manutenibilità e sostituibilità dei componenti.

Nella pratica, però, i layer possono produrre forti dipendenze reciproche, soprattutto quando le interfacce non sono stabili o le astrazioni sono incomplete.

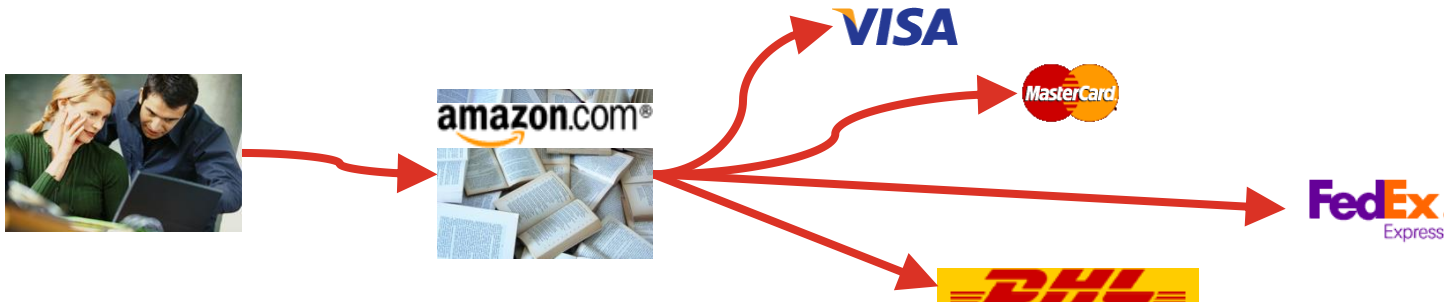
Si è quindi affermato uno stile basato su una **collezione di entità indipendenti**, eseguite in processi separati, che interagiscono tramite **contratti espliciti** (API, messaggi).

L'obiettivo è promuovere **basso accoppiamento, autonomia evolutiva e deploy indipendente** dei componenti.



Services Computing: un cambio di prospettiva

- All'inizio degli anni 2000 si è avvertita la necessità di un cambiamento nella progettazione, nell'implementazione e nella manutenzione dei sistemi rispetto alle architetture distribuite classiche (applicazioni basate sul web).
- **I sistemi tradizionali (distribuiti)** sono sistemi singoli sviluppati in modo top-down mediante decomposizione.
 - Quasi sempre implementano il modello client-server
 - **Accoppiamento forte**: Un'unica organizzazione mantiene e/o possiede il sistema
- **I sistemi distribuiti contemporanei** includono sottosistemi di terze parti
 - Ad esempio, un sistema bancario non offre solo prodotti bancari ai clienti. Offre contratti di assicurazione ...
 - **Accoppiamento debole**: ogni componente viene mantenuto indipendentemente dagli altri.
 - Ogni componente software deve essere in grado di ***servire sistemi diversi in modi diversi***.



Dagli oggetti remoti ai Servizi

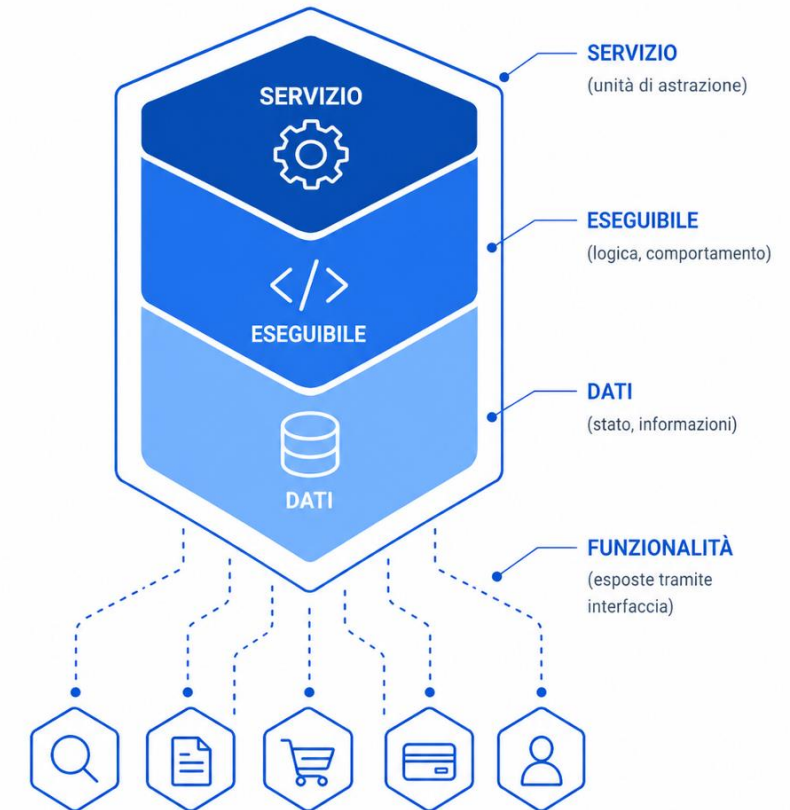
Per capire cos'è concretamente un **servizio** conviene partire da un'idea familiare: l'oggetto della programmazione orientata agli oggetti.

Un oggetto **incapsula uno stato**, espone un'**interfaccia pubblica** e collabora con altri oggetti tramite **scambio di messaggi** (chiamate di metodo).

Un servizio adotta gli stessi principi, ma li proietta su scala distribuita: vive in un processo proprio, su una macchina propria, raggiungibile attraverso la rete.

Le **chiamate a metodo** diventano **invocazioni remote**, l'**interfaccia** diventa un **contratto formale**, e i messaggi viaggiano in formato testuale e platform-neutral, attraverso protocolli standard del web.

Da questa esigenza, all'inizio degli anni 2000, nasce la prima famiglia di tecnologie pensata per i servizi: lo stack **Web Services**, basato su **SOAP** per il messaggio e **WSDL** per descrivere il contratto.



Contratto formale

Descrizione **machine-readable** di cosa il servizio offre e come va invocato: operazioni esposte, tipi di dato in ingresso e in uscita, formato dei messaggi, endpoint di rete.

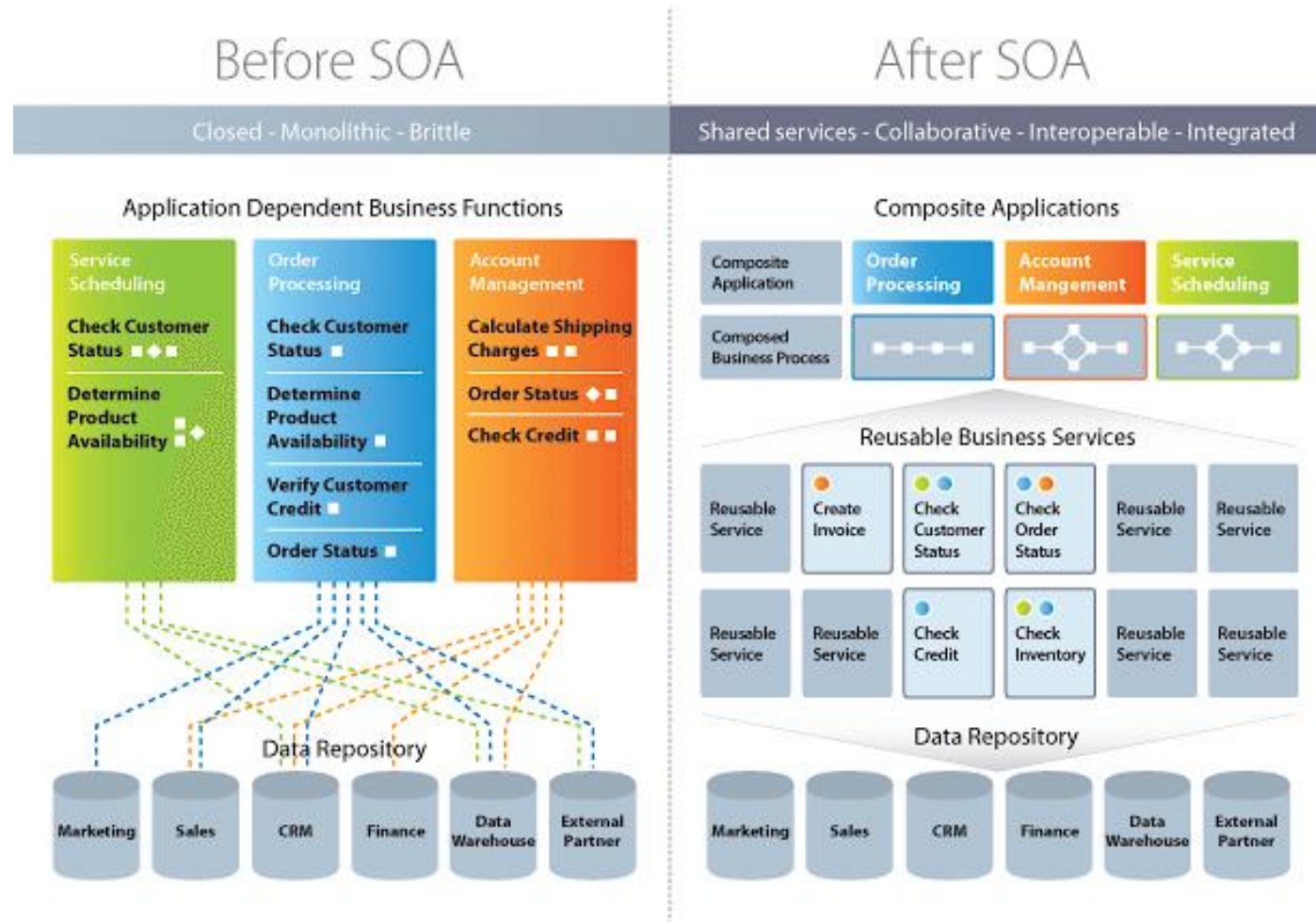
Permette a client e servizio, scritti in linguaggi diversi, di accordarsi senza ambiguità — e di generare automaticamente il codice di chiamata.

Architettura orientata ai servizi - SOA

SOA è uno stile architettonico che si concentra su elementi discreti riutilizzabili (chiamati servizi), invece che su un design monolitico, per costruire le applicazioni.

Un servizio fornisce funzionalità ai richiedenti (che possono essere altri servizi).

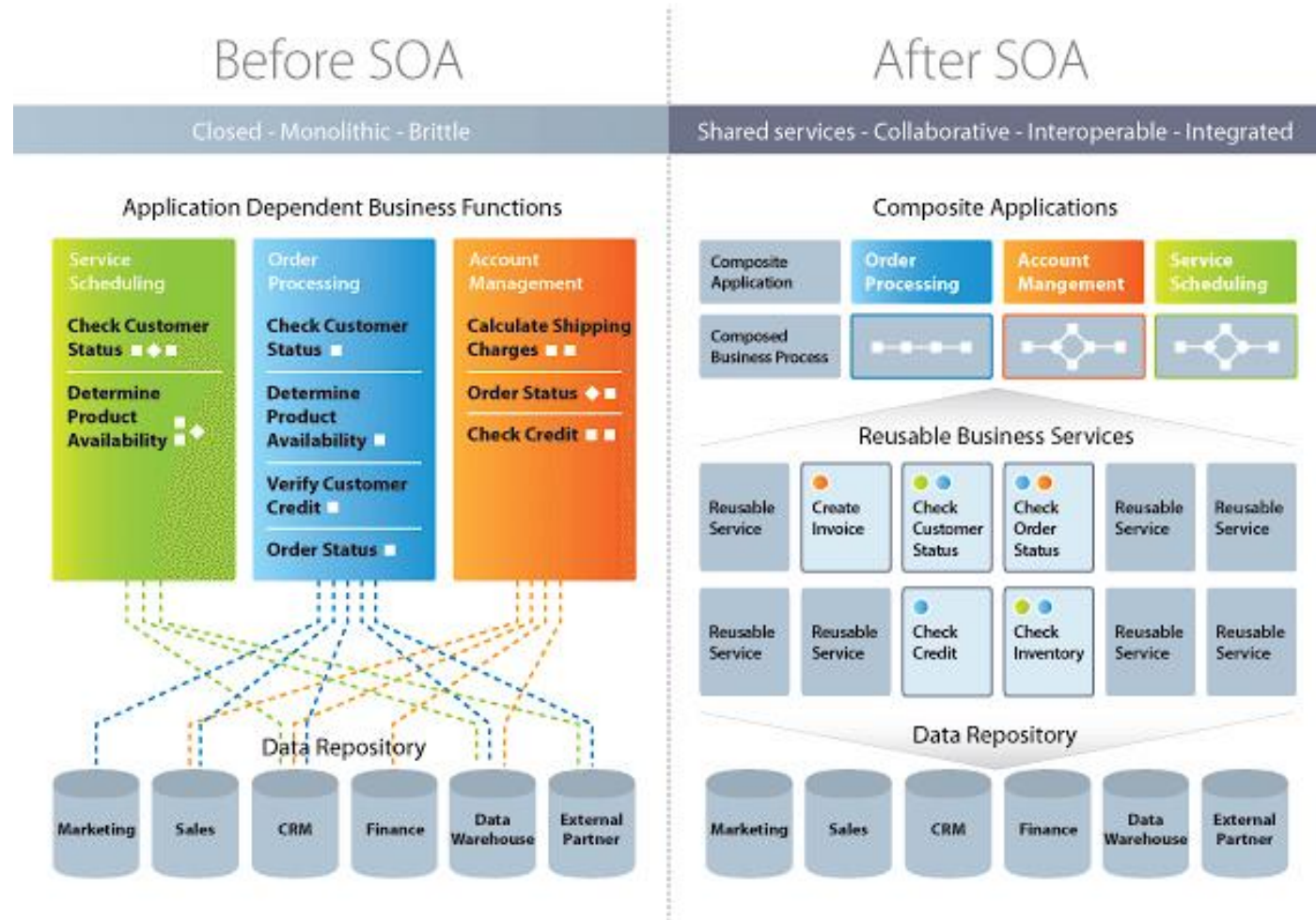
L'accesso al servizio è fornito attraverso la rete (anche Internet).



<https://medium.com/@SoftwareDevelopmentCommunity/what-is-service-oriented-architecture-fa894d11a7ec>

Architettura orientata ai servizi - SOA

- Vantaggi:
 - Riutilizzabilità
 - Processo di sviluppo agile e orientato al business
 - Economia dei servizi
 - Scalabilità
 - Ottimizzazione e riduzione dei costi
- Contro.
 - Gestione del ciclo di vita complesso
 - Dependency Hell
 - Integrazione con le soluzioni legacy
 - ...

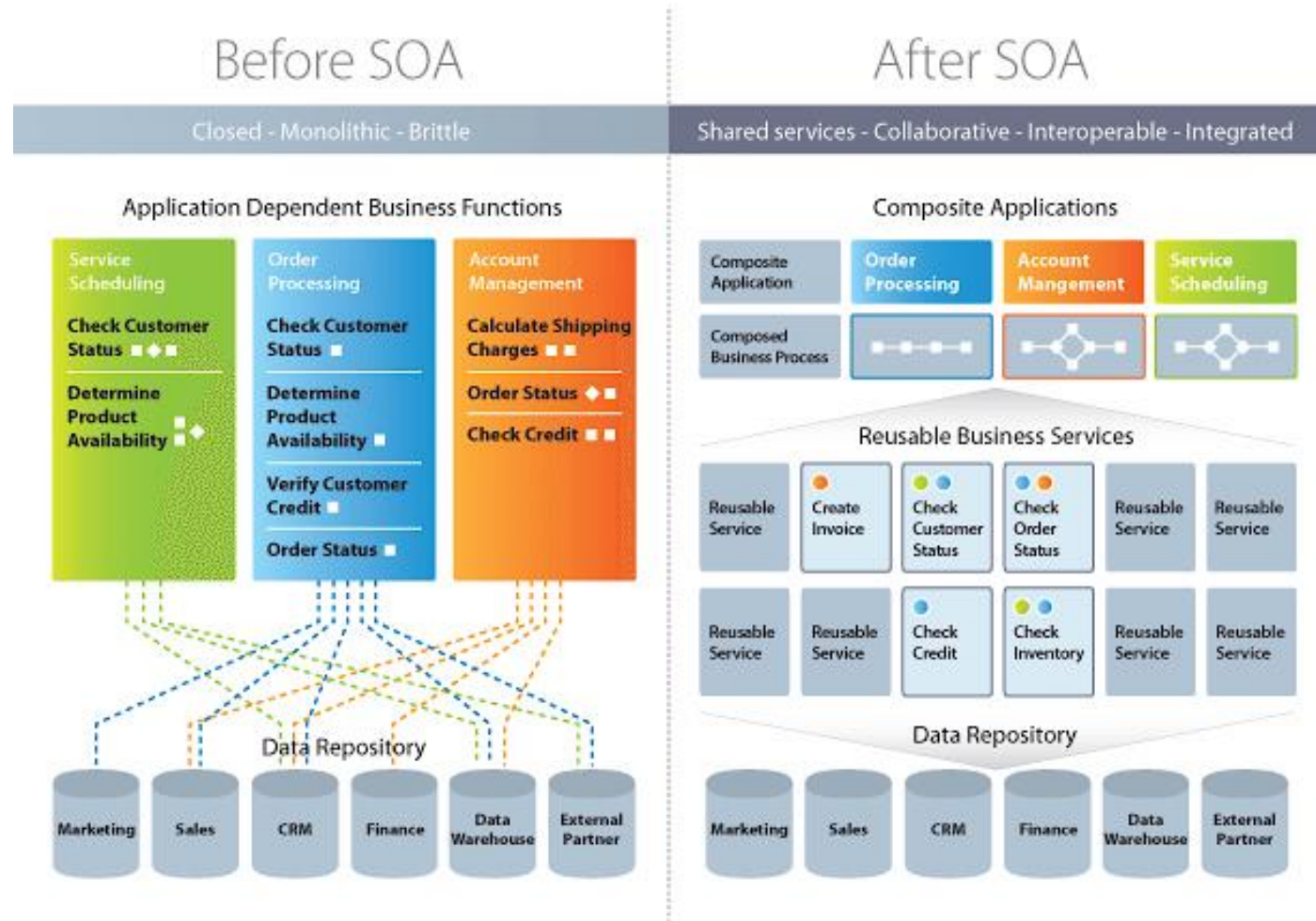


<https://medium.com/@SoftwareDevelopmentCommunity/what-is-service-oriented-architecture-fa894d11a7ec>

Architettura orientata ai servizi - SOA

Ogni servizio deve:

1. Fornire una descrizione delle sue **funzionalità** da **scoprire** e **selezionare**
2. Fornire l'accesso alle sue funzionalità attraverso **protocolli di rete** noti.
3. Supporto alla composizione con altri servizi per fornire soluzioni complesse
4. Rispondere alle esigenze aziendali dei clienti e ai **requisiti** del dominio (funzionali e non funzionali)
5. Garantire un livello di qualità di servizio (QoS)



<https://medium.com/@SoftwareDevelopmentCommunity/what-is-service-oriented-architecture-fa894d11a7ec>

Esempio: SOA in Uber

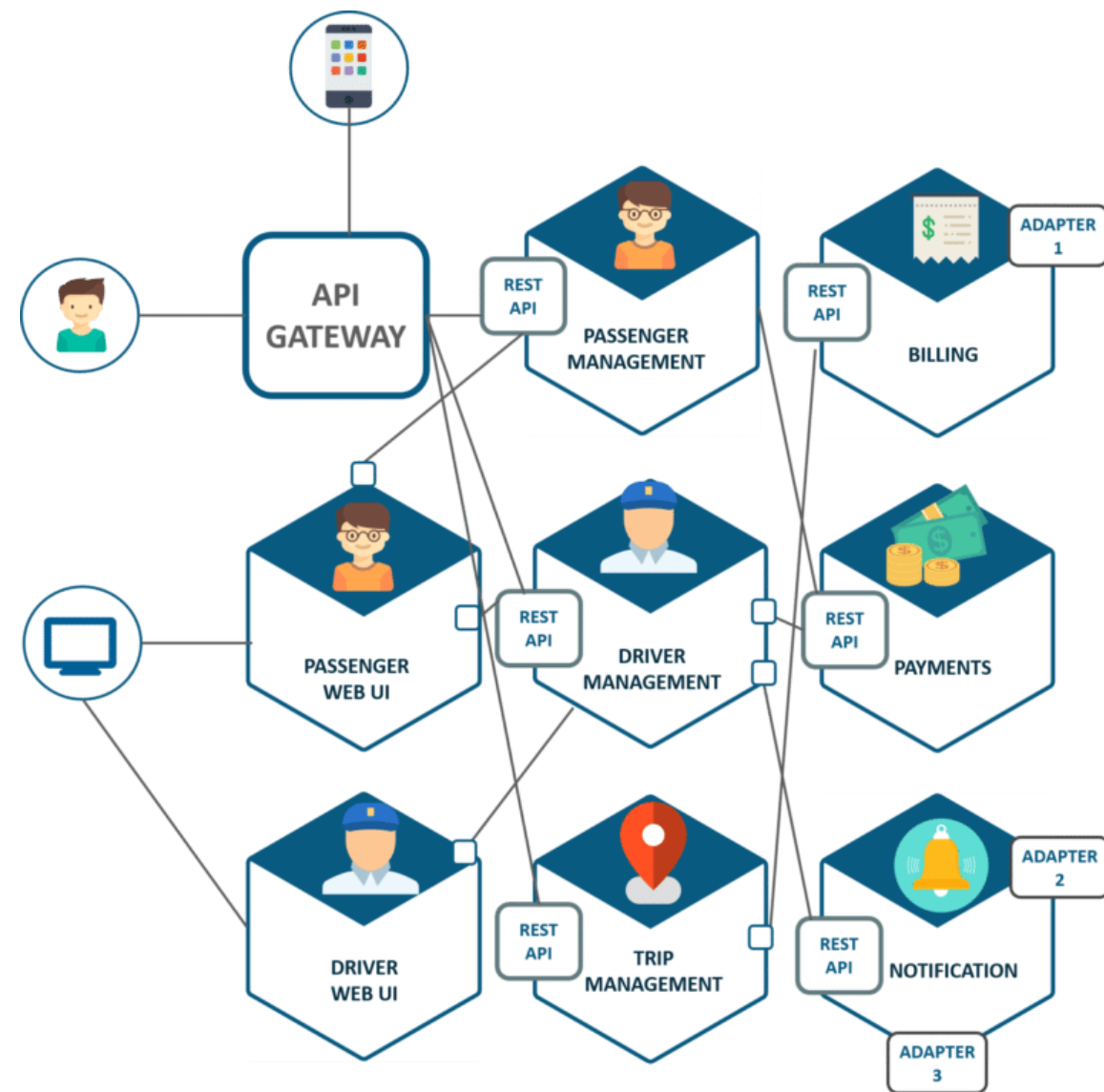
- Architettura SOA in Uber (semplificata)
- I servizi di Uber sono principalmente interni, ma non esclusivamente.
- I servizi implementano un'interfaccia nota (internamente)

Struttura centrale: Un API Gateway che gestisce tutte le comunicazioni tra i componenti e gli utenti esterni

Interfacce utente: Due UI separate - Passenger Web UI per i passeggeri e Driver Web UI per gli autisti

Servizi principali:

- Passenger Management: gestione degli account dei passeggeri
- Driver Management: gestione degli account degli autisti
- Trip Management: coordinamento dei viaggi con integrazione di geolocalizzazione
- Billing: gestione della fatturazione
- Payments: elaborazione dei pagamenti
- Notification: sistema di notifiche con vari adattatori (1, 2, 3)



Fonte: <https://dzone.com/articles/microservice-architecture-learn-build-and-deploy-a>

Ma... cos'è un "servizio (Web)"?

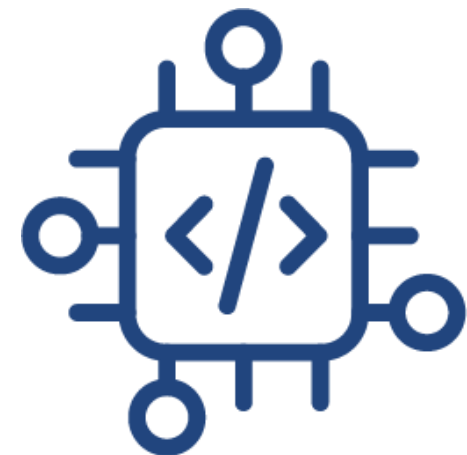
- ! Definizione informale: Un servizio è un'entità software **indipendente** che può essere **scoperta** e **invocata** da altri sistemi software attraverso una rete.

- **Definizione** dell'ente di standardizzazione **W3C**

Un servizio Web è un'applicazione software

1. identificata da un **URI** (Uniform Resource Identifier),
2. le cui **interfacce** [...] sono in grado di essere **definite, descritte e scoperte** [...] e
3. supporta **interazioni dirette** con altri software per mezzo di [...] **messaggi** e **protocolli basati su Internet**.

- Un servizio web utilizza le tecnologie web (in particolare l'HTTP) per fornire funzionalità



Che cos'è un "servizio web"?

Altre definizioni:

- I servizi web sono
 - **autonomo**,
 - **autodescrittivo**,
 - applicazioni modulari che possono essere **pubblicate**, **localizzate** e **invoke** attraverso il Web. (**Tidwell, IBM, 2001**)
- I servizi web sono
 - **incapsulato**,
 - "componenti" Web **liberamente accoppiati** che
 - possono **legarsi dinamicamente** l'uno all'altro. (**Curbera, IBM, 2001**)

Web API

- I servizi sono "**componenti**" indipendenti
 1. Interfaccia nota
 2. Punto di accesso unico
 3. Scambio di dati basato su documenti

Le Web API (Application Programming Interface) sono interfacce che permettono la comunicazione tra applicazioni software attraverso il web, seguendo i principi dei servizi come componenti indipendenti.

1. **Interfaccia** ben nota

- Spesso supportata da un linguaggio di descrizione standard (ad esempio, **WSDL o OpenAPI**)
- Autoesplicativa: Include endpoints, metodi, parametri e risposte
- Contratto di servizio: Definisce chiaramente le interazioni possibili
- Possibile gestione automatica da parte del **middleware**

Il termine **middleware** viene utilizzato soprattutto per il software che consente la comunicazione e la gestione dei dati nelle applicazioni distribuite.

Web API

- I servizi sono "componenti" indipendenti
 1. Interfaccia nota
 2. Punto di accesso unico
 3. Scambio di dati basato su documenti

Le Web API (Application Programming Interface) sono interfacce che permettono la comunicazione tra applicazioni software attraverso il web, seguendo i principi dei servizi come componenti indipendenti.

2. Punto di accesso unico

- Indirizzabile tramite URI base: Il servizio ha un endpoint principale univoco
- Scopribile: Attraverso registri di servizi o discovery services (simile a **UDDI**)
- Gateway unificato: Singolo punto di ingresso per tutte le funzionalità del servizio
- Esempio: <https://api.spotify.com/v1/> come endpoint base del servizio Spotify,
<https://api.twitter.com/2/> per Twitter API v2

UDDI è uno standard basato su XML per descrivere, pubblicare e trovare servizi web. UDDI è l'acronimo di **Universal Description, Discovery, and Integration**.

Web API

- I servizi sono "**componenti**" indipendenti
 1. Interfaccia nota
 2. Punto di accesso unico
 3. Scambio di dati basato su documenti

Le Web API (Application Programming Interface) sono interfacce che permettono la comunicazione tra applicazioni software attraverso il web, seguendo i principi dei servizi come componenti indipendenti.

3. Scambio di dati **basato su documenti**

- Utilizzo di un formato di rappresentazione standard (ad esempio, XML, JSON).
- Usually stateless: Ogni richiesta contiene tutte le informazioni necessarie
- Esempio: Risposta JSON da OpenWeatherMap API

```
json  
  
{  
  "temp": 22.5,  
  "feels_like": 21.8,  
  "humidity": 65,  
  "wind_speed": 5.2  
}
```

Esempio: openweathermap

Un semplice esempio di servizio web è un'API meteo.

- **Identificato da un URI:** L'API meteo è accessibile tramite un URL, ad esempio <https://api.openweathermap.org/data/2.5/weather>
- **Definita, descritta e scoperta:** L'API fornisce una documentazione che descrive le sue funzionalità, i parametri di input (come il nome della città o le coordinate) e gli output previsti (come la temperatura o le condizioni meteorologiche).
- **Supporta le interazioni attraverso i messaggi dei protocolli Internet:** Le applicazioni comunicano con l'API meteo tramite HTTP o HTTPS. Ad esempio, un'applicazione può inviare una richiesta GET per recuperare i dati meteo attuali.

Esempio di interazione

- **Client Request:** GET https://api.openweathermap.org/data/2.5/weather?q=Berlin&appid=your_api_key
- **Server Response:** Dati in formato JSON contenenti informazioni meteo per Berlino, come ad esempio

```
{  
  "weather": [{"description": "clear sky"}],  
  "main": {"temp": 286.15},  
  "name": "Berlin"  
}
```

API call

```
https://api.openweathermap.org/data/2.5/weather?lat=  
{lat}&lon={lon}&appid={API key}
```

Parameters

lat	required	Latitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
lon	required	Longitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
appid	required	Your unique API key (you can always find it on your account page under the "API key" tab)
mode	optional	Response format. Possible values are <code>xml</code> and <code>html</code> . If you don't use the <code>mode</code> parameter format is JSON by default. Learn more
units	optional	Units of measurement. <code>standard</code> , <code>metric</code> and <code>imperial</code> units are available. If you do not use the <code>units</code> parameter, <code>standard</code> units will be applied by default. Learn more
lang	optional	You can use this parameter to get the output in your language. Learn more

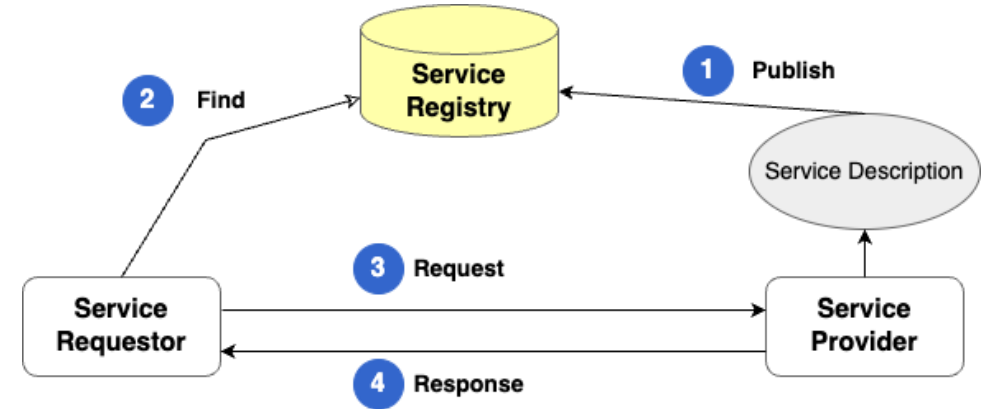
Elementi fondamentali di SOA

Componenti

- **Servizio**: unità funzionale autonoma che esegue una funzione specifica.
- **Descrizione del servizio**: documento che definisce l'interfaccia, le operazioni e i parametri di un servizio.

Ruoli

- **Service Registry**: catalogo che conserva le descrizioni dei servizi, gli endpoint (URL) e le informazioni di binding (protocolli supportati, formati dei dati).
- **Service Provider**: chi crea un servizio e ne pubblica la descrizione nel registry.
- **Service Requestor**: chi consulta il registry per trovare un servizio adatto e lo invoca.

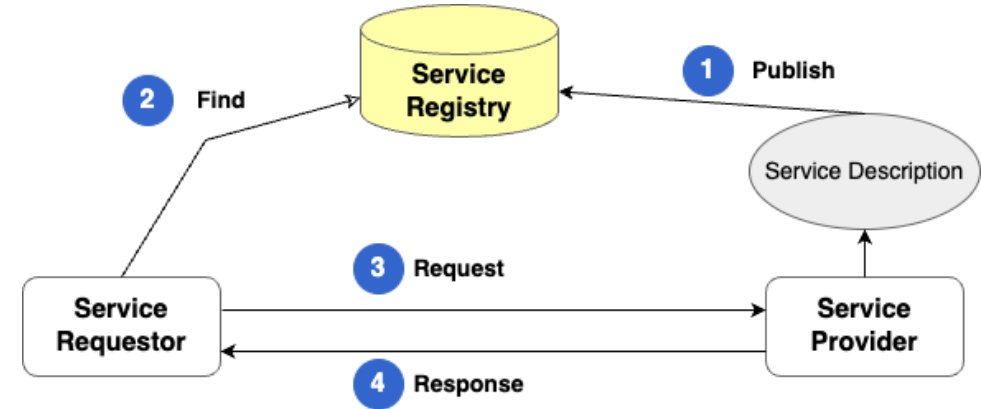


Operazioni di base in SOA

L'interazione tra service provider, requestor e registry si articola in tre operazioni fondamentali.

- **Publish**: il provider pubblica nel registry la descrizione del proprio servizio (operazioni esposte, formati di input/output, endpoint).
- **Find**: il requestor interroga il registry alla ricerca di un servizio compatibile con le proprie esigenze, e ne ottiene la descrizione e l'endpoint.
- **Bind / Invoke**: il requestor stabilisce una connessione diretta con il provider e invoca il servizio (richiesta-risposta).

Il registry partecipa solo alle prime due fasi: una volta ottenuto l'endpoint, requestor e provider comunicano direttamente. È un ruolo passivo da catalogo.



Comunicazione basata su broker

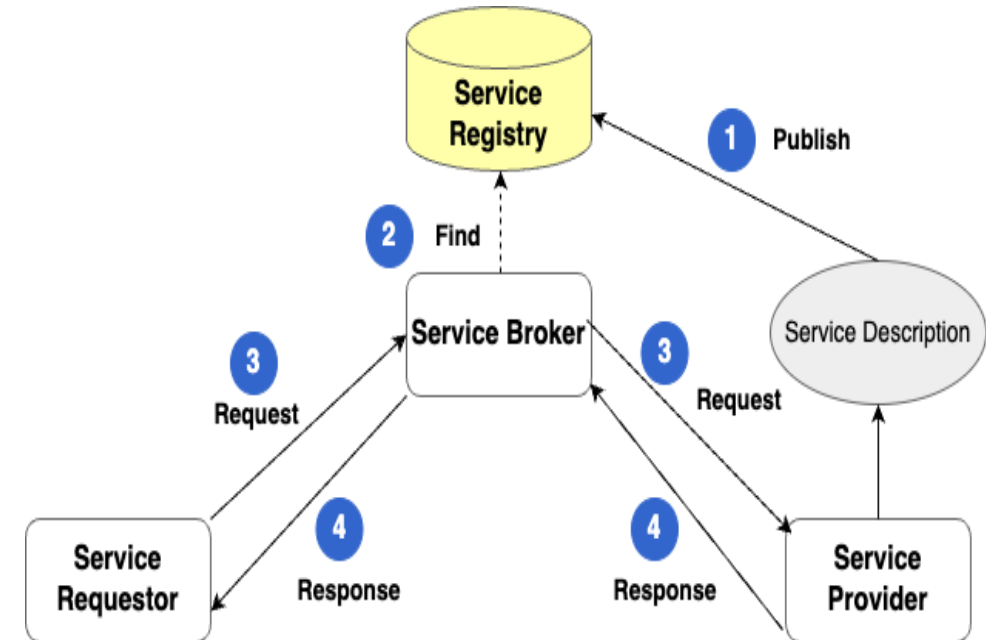
Nel modello classico il registry è passivo: indicizza i servizi ma non partecipa all'invocazione. Quando ai requirements si aggiungono **sicurezza, composizione, mediazione di protocollo** o caching, il registry evolve in **Service Broker**: un intermediario attivo che si interpone in ogni interazione.

Cosa cambia rispetto al registry

- Le richieste passano **attraverso il broker**, non più direttamente al provider.
- Il broker può **autenticare, instradare, convertire** formati e protocolli, comporre più servizi in un'unica chiamata.
- Il **binding diventa dinamico**: il broker sceglie l'istanza del provider al momento della richiesta.

Benefici

- Disaccoppiamento**: provider e requestor non si conoscono direttamente; un provider può essere aggiornato o sostituito senza impatto sui consumatori.
- Interoperabilità**: la mediazione permette a sistemi con protocolli diversi di dialogare.
- Scalabilità**: il broker distribuisce il carico fra più istanze dello stesso servizio.

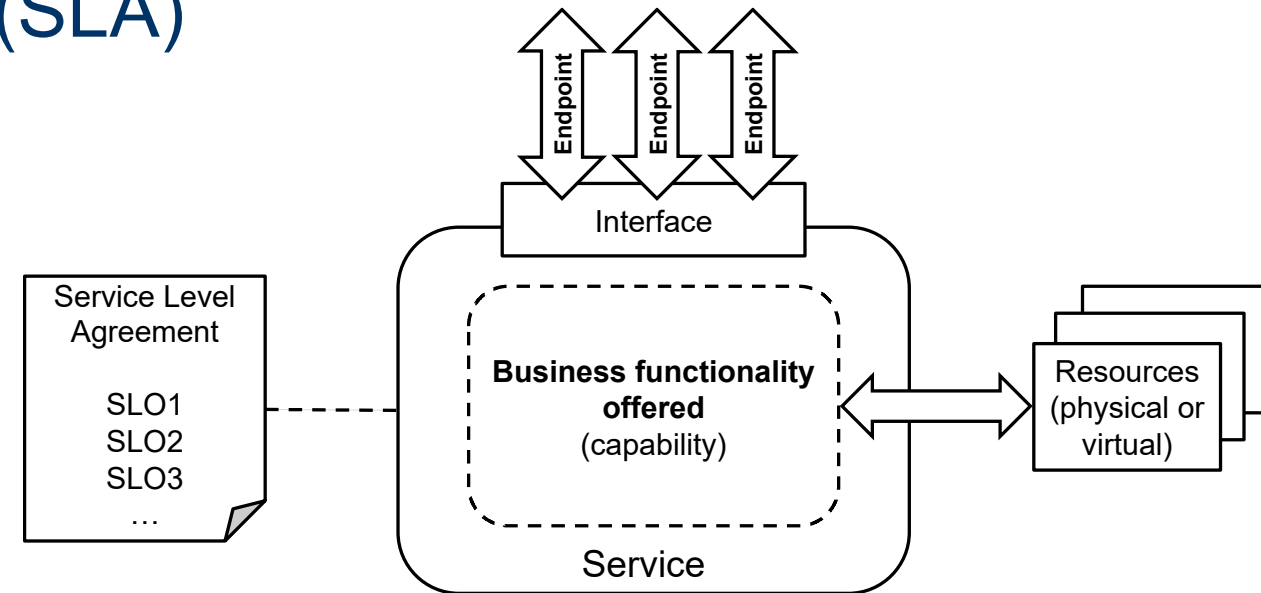


Accordo sul livello di servizio (SLA)

Un **Service Level Agreement (SLA)** è un contratto formalizzato che delinea gli standard, le aspettative e le metriche di prestazione concordate tra un fornitore di servizi e un consumatore di servizi.

Lo SLA definisce le modalità di erogazione di un servizio, piuttosto che limitarsi a descrivere ciò che il servizio comporta.

- **SLA** (Service Level Agreement)
 - Definisce le **caratteristiche non funzionali** garantite dal servizio.
 - Un SLA comprende diversi **SLO** (Service Level Objectives) che definiscono la qualità del servizio da garantire attraverso metriche specifiche.



Examples of SLA Metrics

Metric	Definition	Example Target
Availability	Percentage of time a service is operational.	99.9% per month.
Response Time	Time to acknowledge a reported issue.	15 minutes.
Resolution Time	Time to resolve a critical incident.	4 hours.
Throughput	Volume of transactions processed per second.	1,000 requests/sec.
Error Rate	Percentage of failed transactions.	<1% per month.

SLO

Peculiarità del WS

- Componenti **pubblici**
 - "Scopribili" (naming, registri, accesso)
 - Interfacce pubbliche (protocolli standard, gestiti dalla macchina)
- **Componibilità**
 - Servizi compositi: "orchestrazione"
 - Coordinamento: "coreografia"
- **Descrizioni semantiche**
 - Scoperta
 - Composizione
 - Recommending Systems
 - ...
- **QoS**
 - Base: sicurezza, disponibilità, prestazioni, ...
 - Context awareness
 - ...
- **Organizzazione del sistema**
 - p2p (qualsiasi organizzazione, applicazione gestita)
 - ESB (qualsiasi organizzazione, middleware gestito)
 - Grid (modello dato)

Sfide: come fare...

- **Descrizione**
 - funzionale e QoS
- **Composizione**
 - orchestrazione
 - coreografia
- **Semantica ...**
- **Infrastrutture**
 - Modello architettonico astratto/concreto
 - Servizi per servizi
 - Gestione dei servizi
 - Servizio di denominazione globale
- **Ingegneria del software**
 - Processo di sviluppo
 - Distribuzione
- **Servizi per servizi**
 - Servizio di scoperta globale
 - La reputazione
 - Transazione (flusso di lavoro ed eccezioni)
 - Sicurezza
 - Monitoraggio
- **Business**
 - Supporto ai processi aziendali
 - Modelli economici
 - Modelli organizzativi

Composizione dei servizi

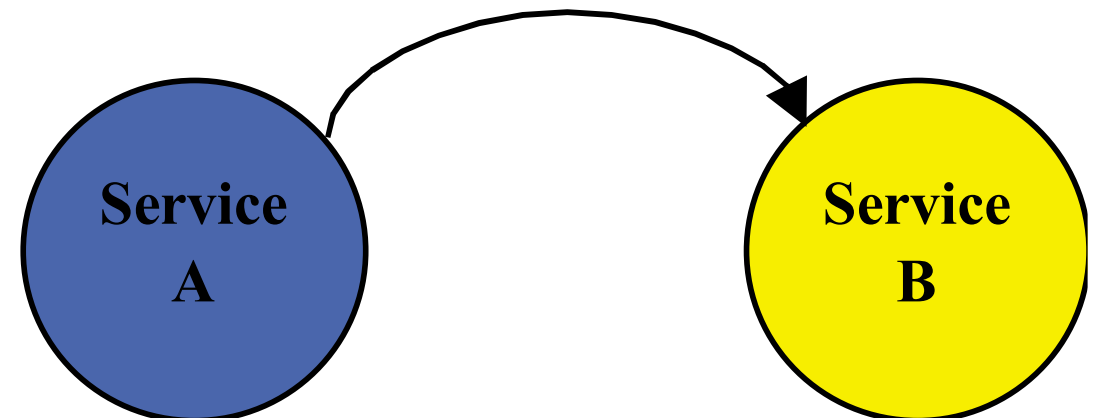
Una **composizione** è un insieme di servizi interconnessi che, presi nel loro insieme, costituiscono a loro volta un nuovo servizio, riutilizzabile in altre composizioni.

Due servizi sono interconnessi quando almeno uno dei due richiama la funzionalità esposta dall'altro attraverso la sua interfaccia (endpoint, API).

Perché la composizione funzioni, è necessaria la **compatibilità** tra i servizi: i servizi devono parlare la stessa lingua sia sintatticamente (stessi formati di messaggio, stessi tipi di dato) sia semanticamente (stesso significato dei dati scambiati).

Per questo ogni provider deve esporre un'**interfaccia pubblicata**, descritta in modo formale e accessibile a chi vuole consumarla.

*Quando una composizione esprime un **flusso di lavoro** coordinato — con un ordine di esecuzione, condizioni e dipendenze tra i servizi — parliamo di **processo di business**.*

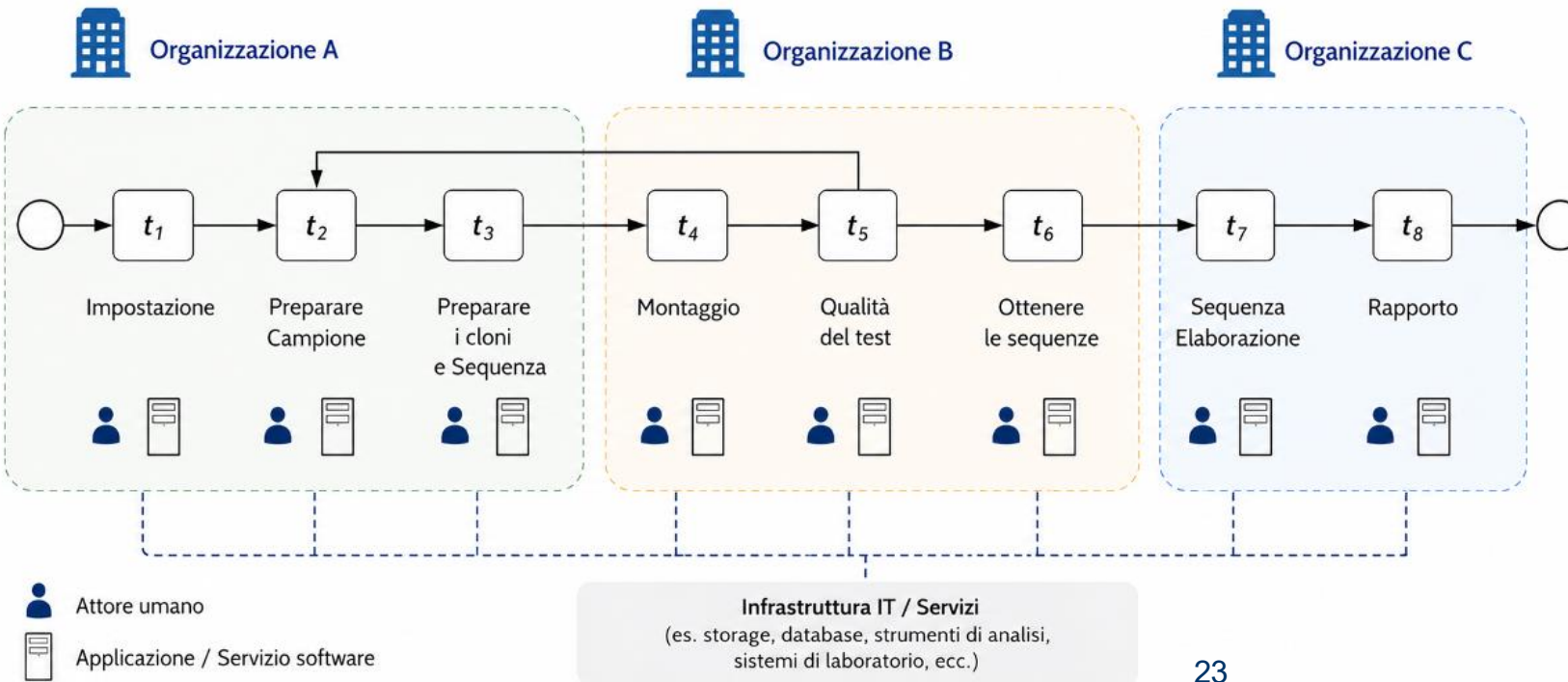


Business Processes

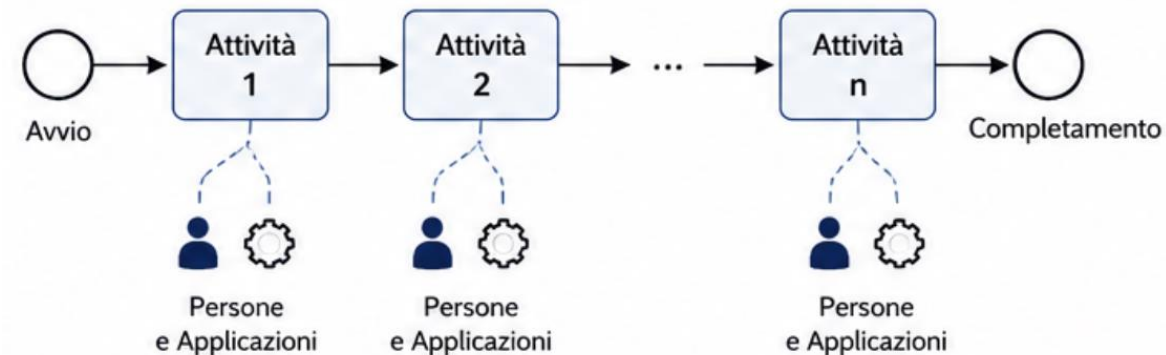
Un **business process** è un workflow strutturato di attività correlate, eseguite da **persone e applicazioni**, con l'obiettivo di ottenere un **risultato ben definito**.

I processi vengono realizzati tramite **composizione di servizi**, in cui ogni attività è implementata come un servizio software coordinato all'interno di un flusso.

Esempio: processo di sequenziamento del DNA (Cardoso 2004)



Schema concettuale



- Può contenere condizioni definite che ne determinano l'avvio e uscite definite al suo completamento.
- Può comportare interazioni formali o relativamente informali tra i partecipanti (umani e software).
- Può contenere una serie di attività automatizzate e/o manuali.
- È distribuito e personalizzato al di là dei confini all'interno delle organizzazioni e tra di esse, e spesso si estende a più applicazioni con piattaforme tecnologiche diverse.
- Ha una durata che può variare notevolmente.
- Di solito è di lunga durata. Una singola istanza può durare mesi o addirittura anni.

Orchestrazione vs coreografia



Orchestrazione: Descrive come i servizi interagiscono tra loro, governati dalla **logica di business** e da un preciso **ordine di esecuzione** dal punto di vista e **sotto il controllo di un singolo attore** (servizio orchestratore).

- Ad esempio, si ha orchestrazione quando un servizio utilizza le funzionalità di altri servizi (nel giusto ordine) per implementare la propria logica di business.

Caratteristiche principali:

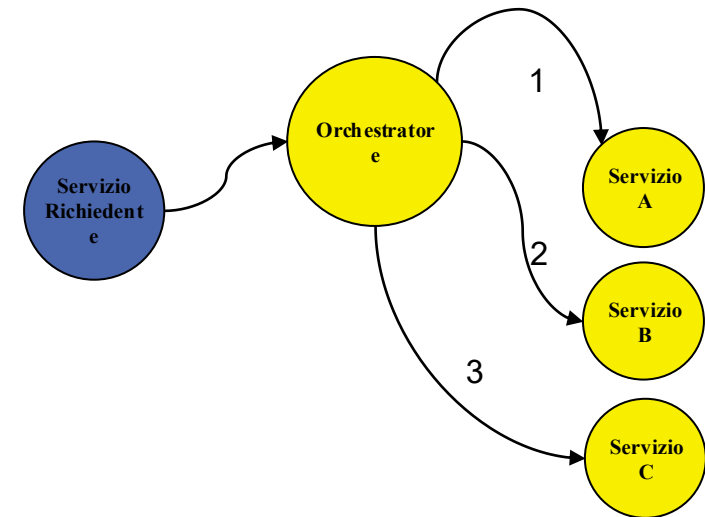
- Controllo centralizzato:** Un servizio orchestratore decide cosa succede e quando
- Gestione esplicita:** L'orchestratore conosce tutti i dettagli del flusso
- Analogia:** Come un direttore d'orchestra che guida tutti i musicisti

Vantaggi:

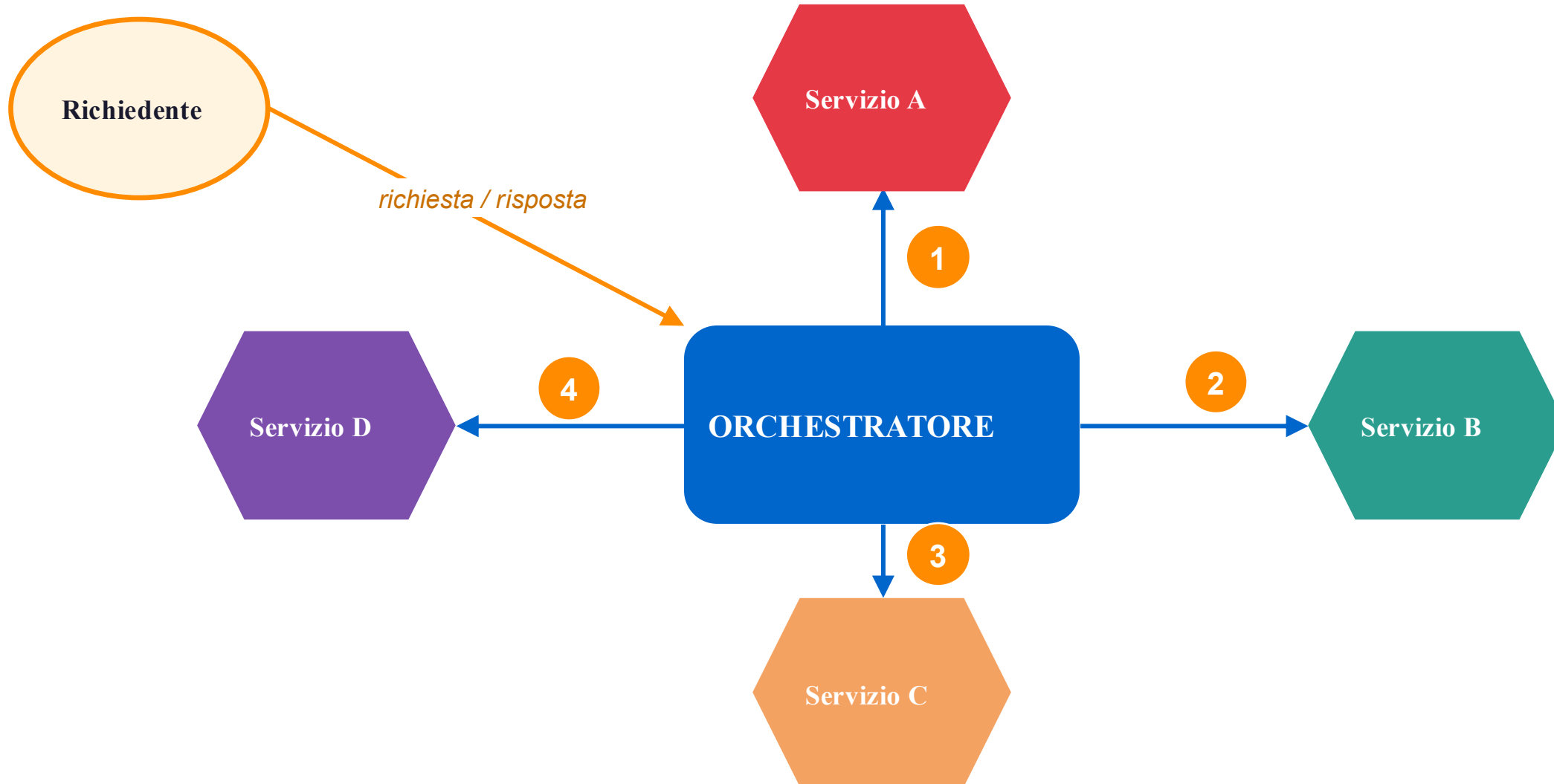
- Monitoraggio semplificato (tutto passa dall'orchestratore)
- Più facile gestire errori e transazioni

Svantaggi:

- Dipendenza dall'orchestratore (single point of failure)
- Accoppiamento più stretto tra servizi
- Scalabilità limitata dall'orchestratore



Orchestrazione — schema



L'orchestratore conosce e governa il flusso: invoca i servizi nel giusto ordine, riceve le risposte, decide cosa fare dopo. Il controllo è centralizzato, i servizi non si parlano direttamente.

Orchestrazione vs coreografia

Coreografia: descrive la sequenza di interazioni tra più parti coinvolte nel processo *dal punto di vista di tutte le parti*. Definisce lo *stato condiviso delle interazioni tra le entità*.

- Ogni servizio è responsabile di portare a termine il proprio compito, invocando anche altri servizi. Di solito viene implementato utilizzando eventi.

Caratteristiche principali:

- **Controllo distribuito:** Ogni servizio agisce autonomamente
- **Comunicazione tramite eventi:** Spesso i servizi reagiscono a eventi pubblicati da altri
- **Analogia:** Come ballerini che si coordinano seguendo una coreografia prestabilita

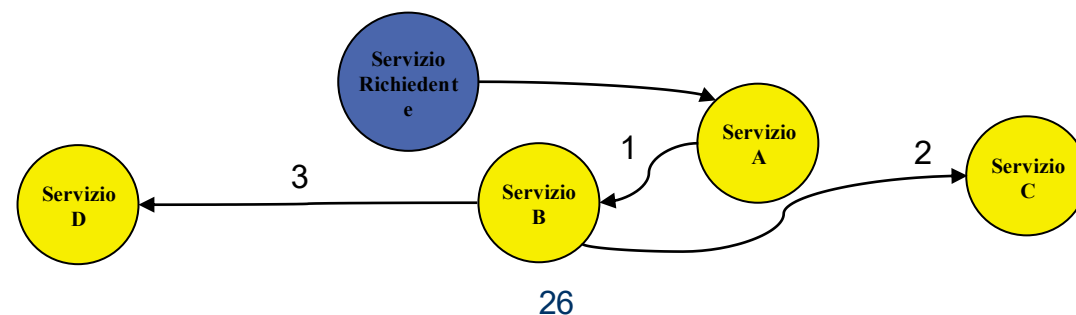


Vantaggi:

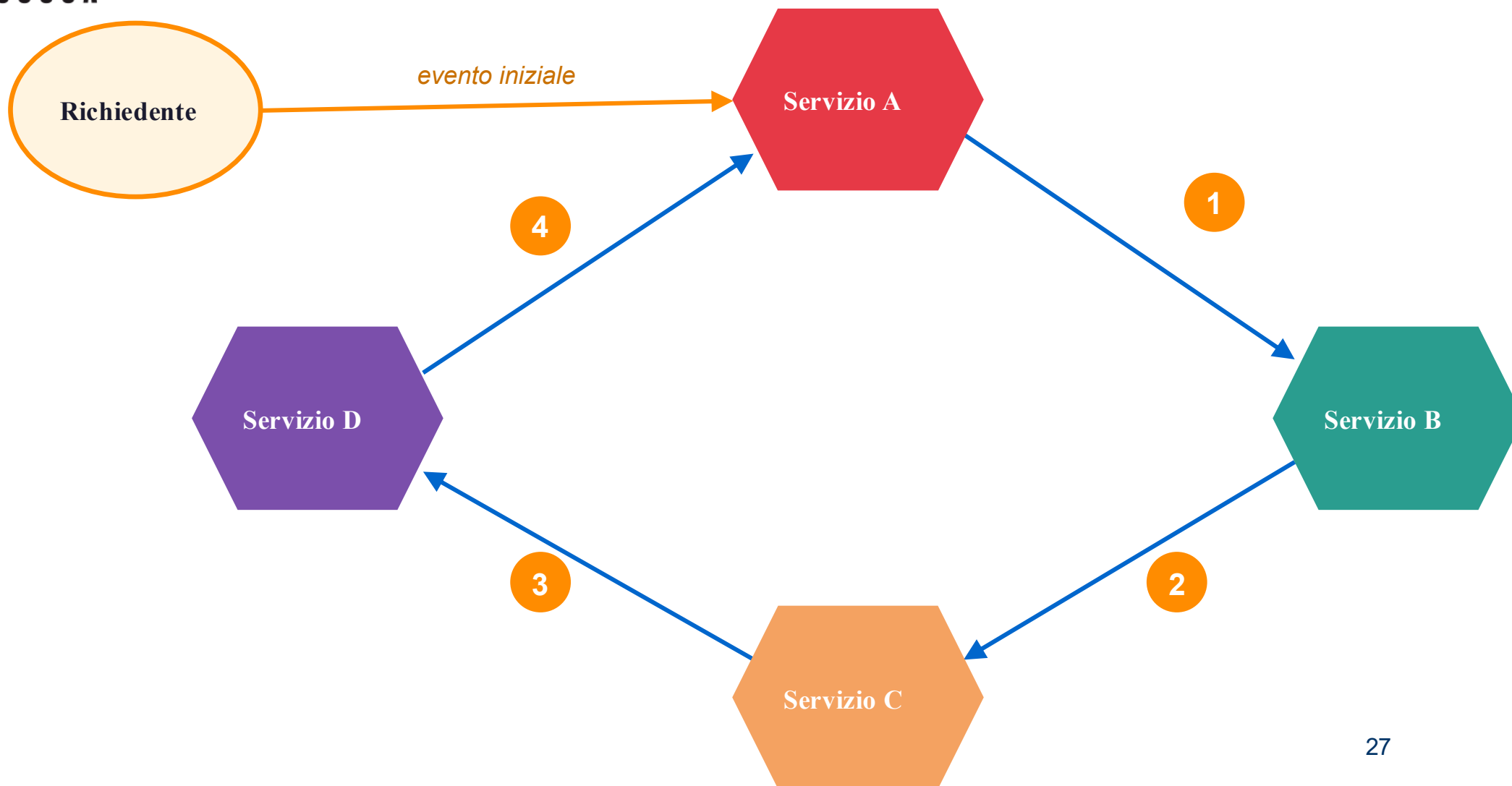
- Alta scalabilità (ogni servizio è indipendente)
- Basso accoppiamento tra servizi
- Resilienza (nessun single point of failure)

Svantaggi:

- Difficile da monitorare (flusso distribuito)
- Gestione errori complessa
- Testing end-to-end più difficile



Coreografia — schema



27

Nessun controllore centrale: ogni servizio sa cosa fare quando riceve un evento e a chi notificare il proprio. Il flusso emerge dalle interazioni; il coordinamento è distribuito.

Orchestratore e broker: due ruoli diversi

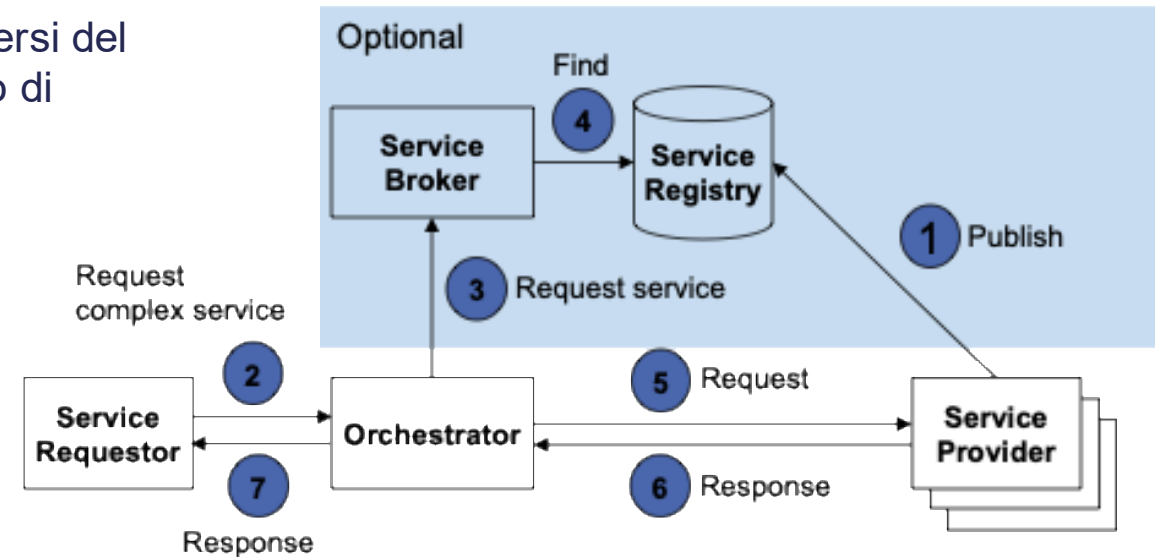
Una composizione di servizi attraverso più organizzazioni richiede qualcuno che la coordini.

SOA distingue due ruoli, spesso confusi, che operano in momenti diversi del ciclo di vita: l'**orchestratore** a tempo di esecuzione, il **broker** a tempo di scoperta.

Orchestratore

Esegue una composizione predefinita

- Conosce in anticipo quali servizi chiamare e in che ordine
- Implementa il **controller** del flusso: invoca, attende risposte, gestisce errori e fallback
- Una sola organizzazione si fa carico del coordinamento per le altre
- Quando: **runtime**, ad ogni



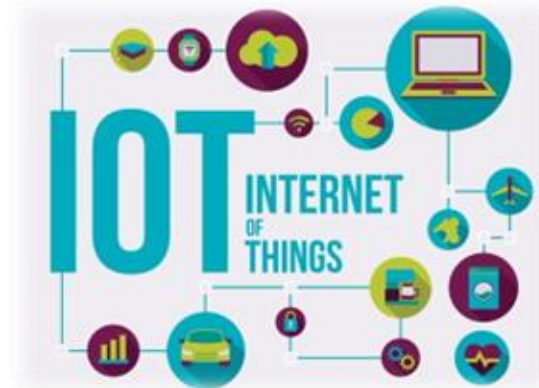
Broker

Trova il servizio giusto al momento giusto

- Non conosce il flusso, ma conosce il **mercato** dei servizi disponibili
- Fa **matchmaking**: il requestor chiede una capacità, il broker indica chi la offre meglio
- Può applicare criteri: prezzo, SLA, reputazione, posizione geografica

IoT e servizi

L'**Internet of Things** (Internet delle cose) è un sistema di oggetti fisici che possono essere *scoperti, monitorati, controllati* o *interagire con* altri dispositivi elettronici, che comunicano attraverso varie interfacce di rete e *possono essere collegati ad Internet*.¹



I dispositivi elettronici sono in grado di raccogliere e scambiare dati, ed eseguire azioni per realizzare le piattaforme del futuro.
=> **servitizzazione** del mondo reale



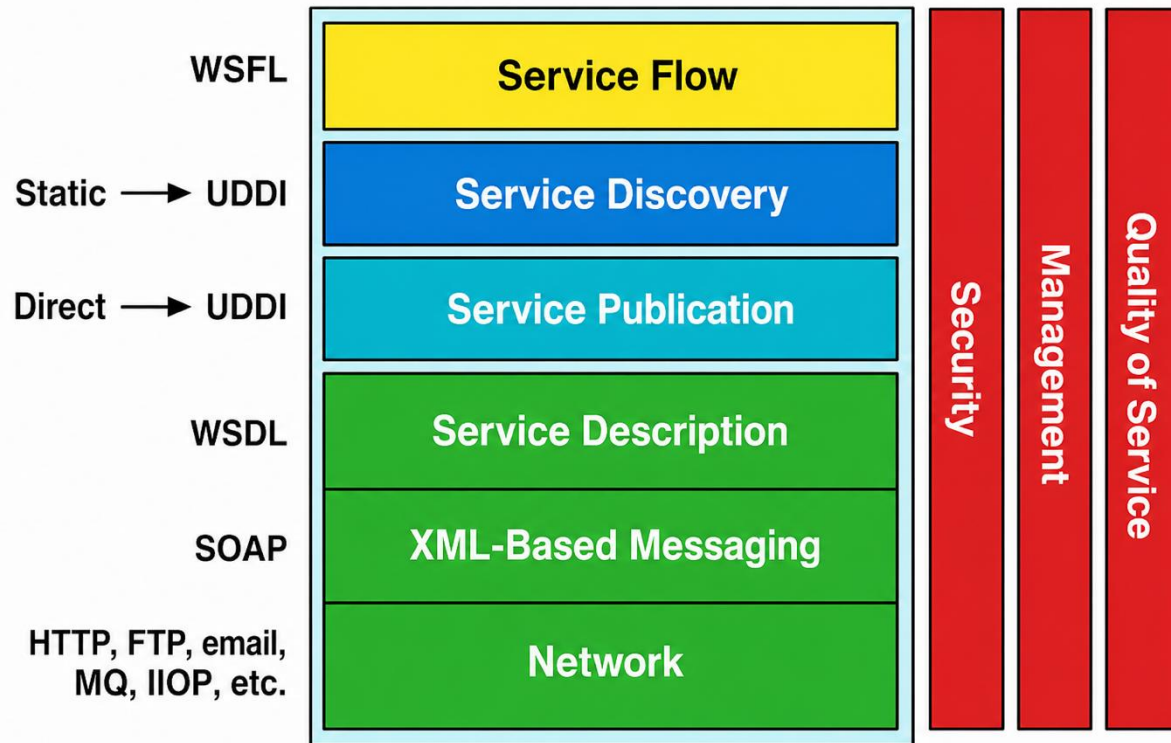
Le cose come servizio

- Elevato numero di componenti coinvolti in singole applicazioni
- (Ri)configurazione dinamica delle applicazioni
- **Le cose consumano/espongono funzionalità (servizi)**

Esempio: Quando un cliente entra in un negozio con uno smartphone, entra a far parte di un *ecosistema* che comprende sensori installati nel negozio, servizi che accedono ai dati del sistema informativo del rivenditore e servizi Internet esterni.

SERVIZI SOAP

Lo stack concettuale dei servizi web (SOAP)



- **Business Process: BPEL/WSFL**

Linguaggio di esecuzione dei processi aziendali

- **Search and Find: UDDI**

Universal Discovery Description and Integration, per i registri dei servizi Web.

- **Description: WSDL**

Web Services Description Language, per descrivere servizi in rete basati su XML.

- **Messaggi: SOAP**

Simple Object Access Protocol, per definire un modo uniforme di passare dati codificati in XML.

- **Network: Protocolli Internet**

HTTP, STMP, ecc.

Attenzione. La tecnologia di questo stack si riferisce esplicitamente ai servizi **SOAP**.

SOAP in breve

SOAP è un **protocollo di scambio messaggi** in formato XML, pensato per far dialogare applicazioni scritte in linguaggi diversi attraverso la rete.

Il problema che doveva risolvere: chiamare procedure remote (RPC) tra sistemi eterogenei — un client Java che invoca un servizio .NET, due aziende che devono integrare i loro gestionali — **senza accordarsi prima sul linguaggio**.

La soluzione: un'**envelope XML standardizzata** che viaggia su qualunque protocollo di trasporto (tipicamente HTTP) e un contratto formale, WSDL, che descrive cosa il servizio offre.

Cronologia in due righe

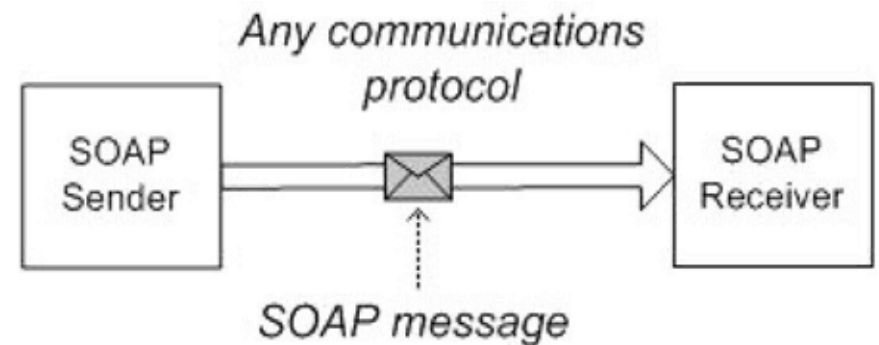
- **1998**: Microsoft propone XML-RPC, evolve in SOAP
- **2003**: SOAP 1.2 standard W3C
- **2003-2008**: età d'oro — banche, telco, integrazioni B2B enterprise
- **dal 2010**: declino, REST diventa lo standard de facto del web

In pratica

Una banca espone un servizio **VerificaSaldo**: il client (Java) costruisce un messaggio SOAP con dentro l'IBAN, lo invia via HTTP POST, riceve un altro messaggio SOAP con il saldo. Stesso identico scambio se il server è in .NET o Python — il contratto WSDL li mette d'accordo.

SOAP: Simple Object Access Protocol

- SOAP è un [protocollo basato su XML](#) che consente ad applicazioni software di comunicare utilizzando messaggi XML.
 - Gli spazi dei nomi XML sono utilizzati per fornire una semantica ai dati (non solo ai tipi di dati).
 - È agnostico rispetto al sottosistema di trasporto.
 - Permette di creare messaggi di richiesta e risposta in diverse configurazioni
- SOAP è un modo [standard](#) per strutturare i messaggi XML (dati).
 - Un'applicazione delle specifiche XML
 - Si affida a XML Schema, spazi dei nomi XML
- Non è legato ad alcun linguaggio o piattaforma di programmazione (nemmeno ai servizi web).
- Estensibile
- [Utilizzato nei servizi orientati ai documenti](#)



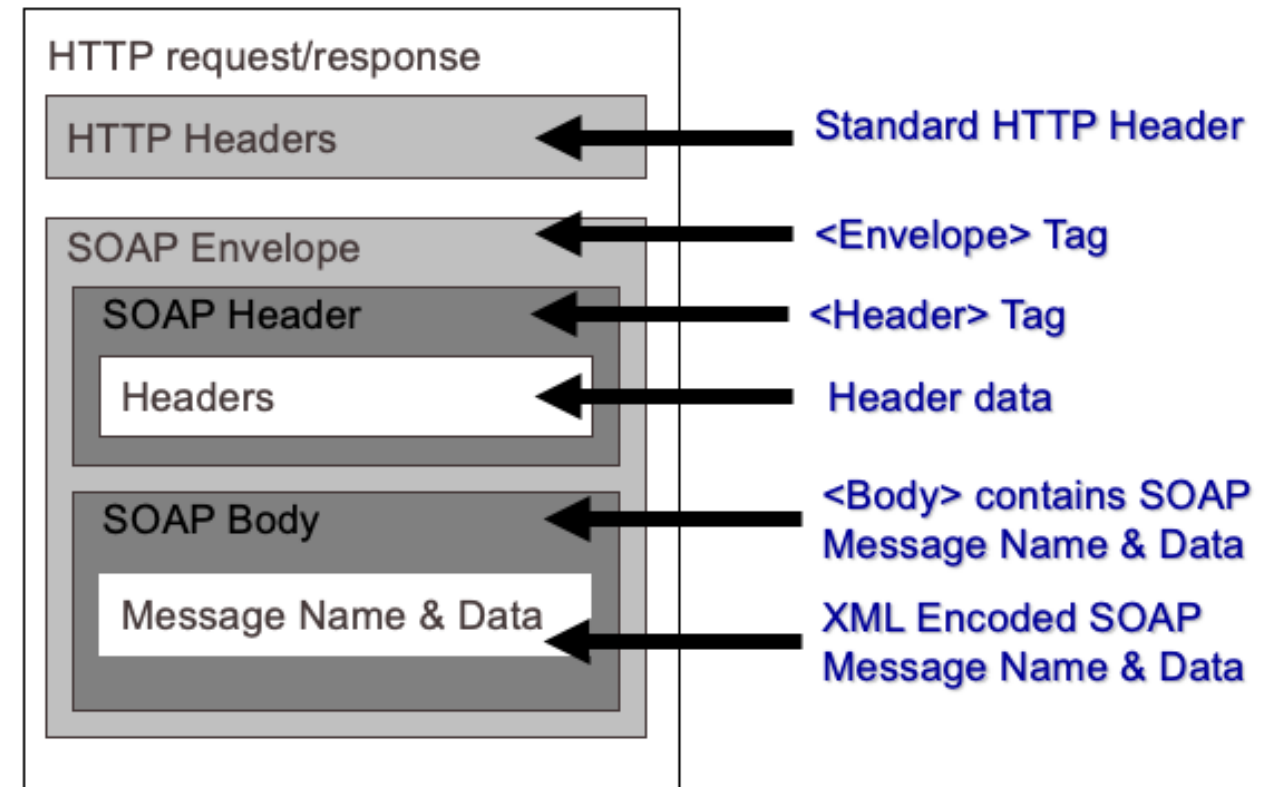
Componenti di un messaggio SOAP



- **SOAP Envelope**
 - Ingloba il contenuto del messaggio

- **SOAP Header** (opzionale)
 - Maggiore flessibilità, può essere elaborato dai nodi tra l'origine e la destinazione.
 - Contiene blocchi di informazioni su come elaborare il messaggio:
 - Impostazioni di instradamento e consegna
 - Asserzioni di autenticazione/autorizzazione
 - Contesti di transazione

- **SOAP Body**
 - Messaggio effettivo da consegnare ed elaborare
 - Sia per le informazioni di richiesta che per quelle di risposta



SOAP con HTTP

- Sebbene SOAP possa utilizzare diversi protocolli di trasporto, HTTP è quello più comunemente utilizzato.
- Le richieste HTTP sono costituite da un metodo, come POST o GET, seguito dall'URL richiesto e dalla versione del protocollo (ad esempio HTTP 1.1).
- Le risposte seguono la semantica di HTTP per fornire il codice di stato della risposta;
 - Ad esempio, un **codice di stato** del tipo 2xx indica che la richiesta è stata ricevuta, compresa, accettata, ecc.
- Esistono due modelli per lo scambio di messaggi SOAP via HTTP:
 - il modello "**SOAP request-response**", in cui il metodo **POST** viene utilizzato per portare i messaggi SOAP nel corpo delle richieste/risposte http;
 - il modello "**SOAP response**", in cui nelle richieste HTTP viene utilizzato il metodo **GET** per ottenere il messaggio SOAP nel corpo della risposta.
 - Utilizzando SOAP con HTTP, bisogna ricordarsi di impostare l'intestazione **Content-Type** come **application/SOAP+xml**.

SOAP con HTTP: SOAP response

- Richiesta HTTP GET

```
GET /travel.example.org/reservations?code=F3T HTTP/1.1
Host: travelcompany.example.org
Accept: text/html;q=0.5, application/soap+xml
```

- Risposta HTTP GET

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml; charset="utf-8"
Content-Length: 200
<?xml version='1.0' ?>
<env:Envelope
  xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header>...</env:Header>
  <env:Body>...</env:Body>
</env:Envelope>
```

SOAP con HTTP: SOAP request-response

- Richiesta HTTP POST

```
POST /Reservations HTTP/1.1
Host: travelcompany.example.org
Content-Type: application/soap+xml; charset="utf-8"
Content-Length: 230

<?xml version='1.0' ?>
<env:Envelope
  xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header> ... </env:Header>
  <env:Body> ... </env:Body>
</env:Envelope>
```

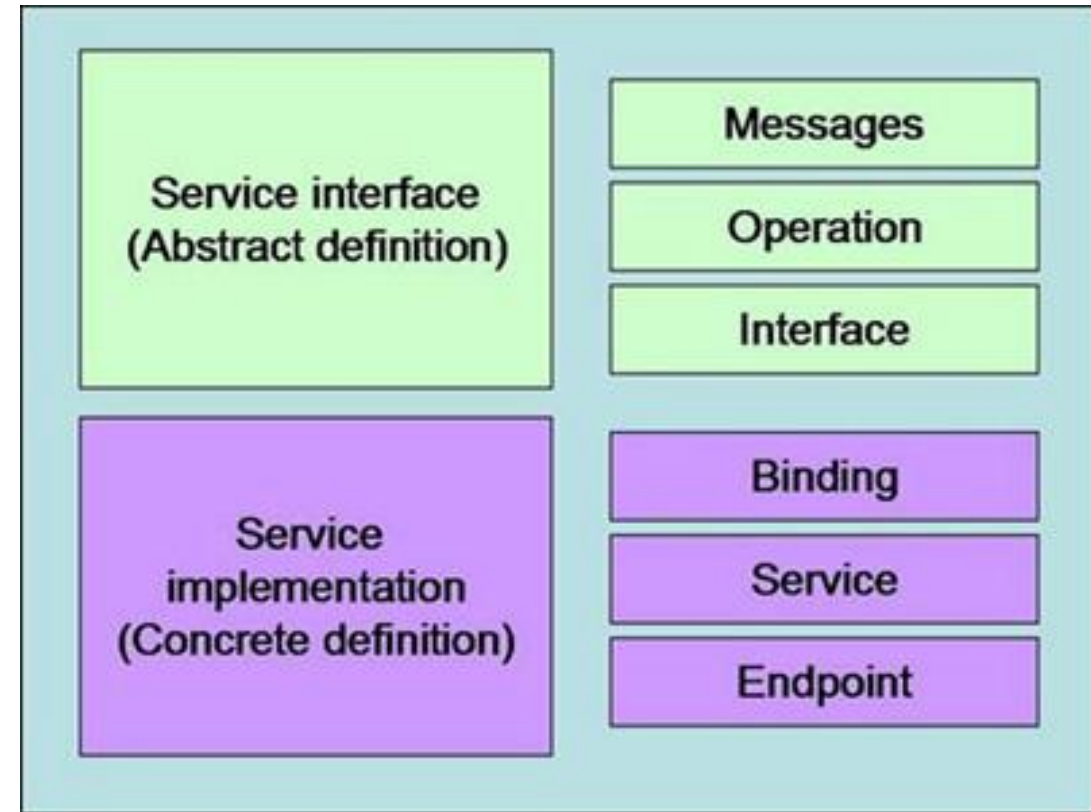
- Risposta HTTP POST
 - Come nell'esempio precedente

Che cos'è WSDL?

- [Linguaggio di descrizione dei servizi web](#) (si pronuncia [wizdel])
 - Utilizzato per definire *"Come e dove accedere al servizio"*.
- [Linguaggio basato su XML](#) per descrivere i servizi web, i messaggi e le modalità di invocazione.
 - Un file WSDL contiene una descrizione dell'interfaccia di un servizio.
- WSDL consente di descrivere quattro dati principali per un servizio:
 - Informazioni sull'[interfaccia](#) che descrivono tutte le operazioni pubblicamente disponibili di un servizio.
 - Dichiarazioni di [tipi di dati](#) per tutti i messaggi. I tipi complessi possono essere dichiarati (utilizzando SOAP) e utilizzati
 - Informazioni di [binding](#) sul protocollo di trasporto (HTTP, SMTP, UDP)
 - Informazioni sull'[indirizzo](#) per la localizzazione del servizio (URI)
- WSDL è una raccomandazione del W3C
 - WSDL 2.0 è diventato una raccomandazione del W3C 26 giugno 2007

WSDL 2.0: modello concettuale

- Nella parte *astratta*
 - Descrizione di un servizio in termini di **messaggi** che invia e riceve attraverso un sistema di tipi, tipicamente *W3C XML Schema*.
 - Un'**operazione** associa modelli di scambio di messaggi a uno o più messaggi.
 - I **modelli di scambio di messaggi** definiscono la sequenza e la cardinalità dei messaggi scambiati tra i nodi (servizi).
 - Un'**interfaccia** raggruppa queste operazioni in modo indipendente dal canale di comunicazione.
- Nella parte *concreta*
 - I **binding** specificano il protocollo di trasporto per le interfacce.
 - Un **endpoint** associa un URI a un binding
 - Un **servizio** raggruppa gli endpoint che implementano un'interfaccia comune



WSDL 2.0: definizione del binding

Binding

- L'attributo **name** definisce il nome del binding. Con questo nome si può fare riferimento ad esso quando si definisce un endpoint di servizio.
- Ogni nome di binding deve essere univoco all'interno dello spazio dei nomi del target WSDL 2.0.
- L'attributo **interface** contiene il nome di una delle interfacce definite
- L'attributo **type** definisce il formato del messaggio da utilizzare. In questo caso è SOAP (sono possibili altri binding).
- **wsoap:protocol** definisce il protocollo di "trasporto". Nell'esempio, i messaggi soap verranno trasportati utilizzando HTTP

```
<binding name = "myServiceInterfaceSOAPBinding"
  interface = "tns:myServiceInterface"
  type = "http://www.w3.org/ns/wsdl/soap"
  wsoap:protocol = "http://www.w3.org/2003/05/soap/bindings/HTTP/">
  <operation ref = "tns:checkServiceStatusOp"
    wsoap:mep = "http://www.w3.org/2003/05/soap/mep/soap-response"/>
  <fault ref = "tns:dataFault" wsoap:code = "soap:Sender"/>
</binding>
```

Operation

- L'attributo **ref** dell'elemento operation fa riferimento a una specifica operazione (già definita nella sezione interface)
- L'attributo **wsoap:mep** definisce lo schema di scambio dei messaggi per SOAP (richiesta GET)

Fault

- L'attributo **ref** definisce quale errore (già definito nella sezione dell'interfaccia) ci si riferisce per questo binding
- L'attributo **wsoap:code** dell'elemento fault definisce il codice di errore che determina l'invio di questo messaggio.

WSDL 2.0: definizione di service/endpoint

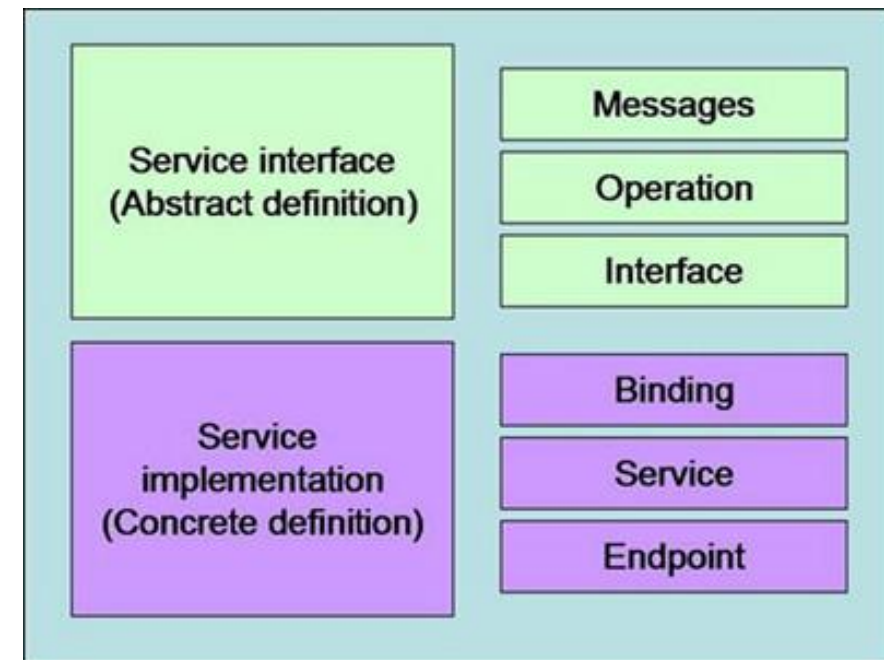
Service

- L'attributo **name** definisce il nome del servizio. Ogni nome di servizio deve essere unico all'interno dello spazio dei nomi di destinazione.
- L'attributo **interface** definisce il nome dell'interfaccia (precedentemente definita nella sezione interface) implementata dal servizio.

```
<service name = "myService" interface = "tns: myServiceInterface">  
  <endpoint name = "myServiceEndpoint"  
    binding = "tns:myServiceInterfaceSOAPBinding"  
    address = "http://yoursite.com/MyService" />  
</service>
```

Endpoint

- L'attributo **name** definisce il nome dell'endpoint, che deve essere unico all'interno dei servizi.
- L'attributo **binding** specifica quale binding già definito verrà utilizzato da questo endpoint.
- L'attributo **address** specifica l'indirizzo fisico dove il servizio sarà disponibile.



Perché il sogno di SOAP è svanito

Obiettivo originale: Creare uno standard universale per la comunicazione tra applicazioni

Promessa: Indipendenza da piattaforme, linguaggi e protocolli di trasporto

I fattori del declino

Complessità eccessiva

- Struttura XML verbosa e rigida
- Curve di apprendimento ripide
- Costi di implementazione elevati

Performance insufficienti

- Overhead significativo nella trasmissione dei dati
- Parsing XML computazionalmente costoso
- Latenza più elevata rispetto alle alternative

Ascesa delle alternative

- REST: Più semplice, leggibile, più veloce,
- JSON: Formato dati leggero e più facile da elaborare

Eredità di SOAP

- Vivo dove serve affidabilità forte: banche, pagamenti, integrazioni B2B legacy
- Ha lasciato standard ancora in uso (WS-Security, transazioni distribuite)
- Idea del **contratto formale** (WSDL) ripresa da OpenAPI, gRPC e GraphQL
- Lezione: la composizione di servizi resta valida; cambiano formato, protocollo e stile